# Grade Calculator Example in JavaScript

JavaScript + HTML Code

# Understand MVC Architecture

- Model: Data Model
- View: User Interface
- Controller: Application Logic (Also called Business Logic)

HTML, CSS, and JavaScript separate *technologies* (structure, presentation, behavior). That is different from separating the MVC *roles*.

# Data Model, UI, and Application Logic

Also, when we design software we need to identify the "Data Model", "User Interface", and "Application Logic".

- Data Model: The data structure that all of the application components should share
- User Interface: The components that users use to interact with the application
- Application Logic: The code that processes data and updates the UI

## Divide and Conquer

We need to isolate these elements:

- We isolate UI elements anytime necessary (we call this View).
- We isolate Data Model so that it can be easily extended and maintained (We can support API to access the core data structure).
- We isolate application logic so that it can be tested and changed without touching the UI (separation of concerns).

# MVC Isolation

- Model for data representation
- View (HTML) for displaying information
- Control for application (or business) logic

## Model (Data Structure)

```
// Grade Calculator Application
let grades = [];
```

# View (HTML)




- Three inputs and one button

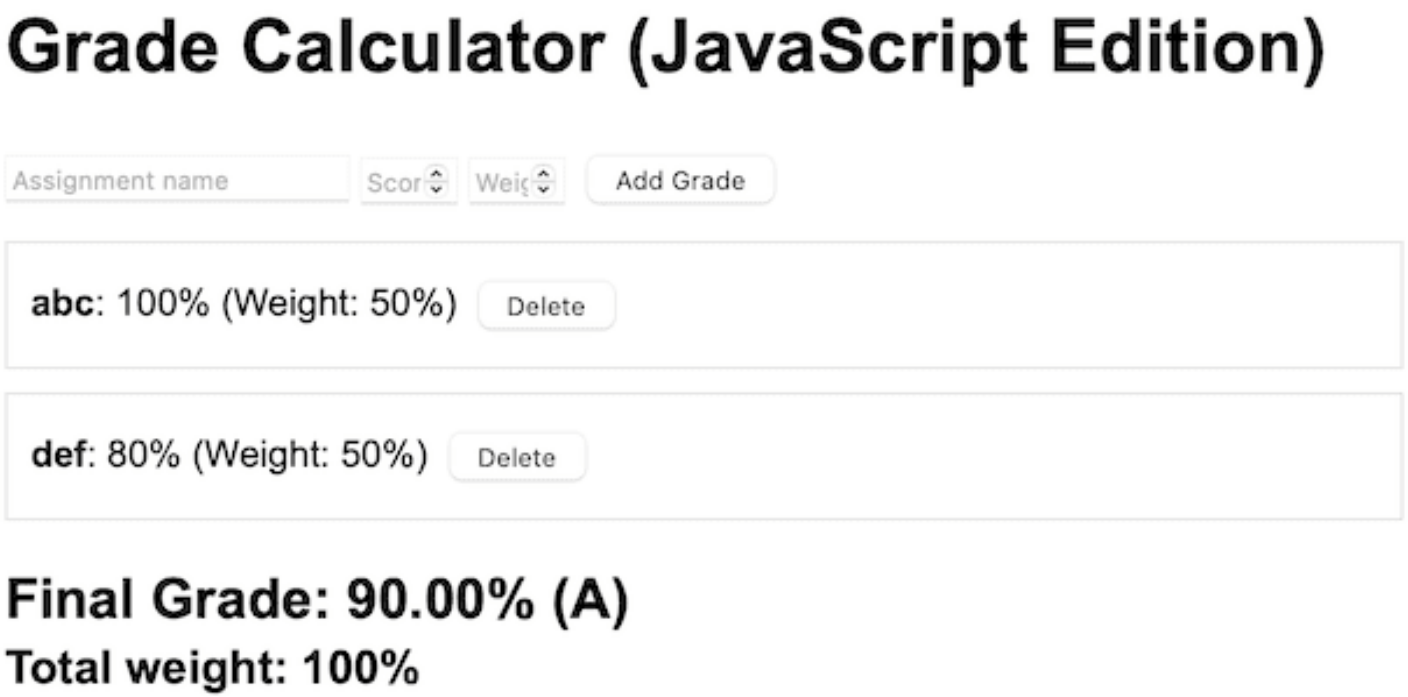
```
<h1>Grade Calculator (JavaScript Edition)</h1>

<div id="grade-form">
  <input type="text" id="assignment-name" placeholder="Assignment name" />
  <input type="number" id="score" placeholder="Score" min="0" max="100" />
  <input type="number" id="weight" placeholder="Weight %" min="0" max="100" />
  <button onclick="addGrade()">Add Grade</button>
</div>

<div id="grades-list"></div>
<div id="result" class="result"></div>

<script src="app.js"></script>
```

With two grades, 100 (50%) and 80 (50%), the final value is calculated and displayed: 100 x 0.5 + 80 x 0.5 = 90 (A)



## Application Logic (Business Logic)

In this level, we process the data to display the processed results.

```
// Add a new grade
function addGrade() {
  // Get input values
  const nameInput = document.getElementById('assignment-name');
  const scoreInput = document.getElementById('score');
  const weightInput = document.getElementById('weight');

  const name = nameInput.value.trim();
  const score = parseFloat(scoreInput.value);
  const weight = parseFloat(weightInput.value);

  // Validation
  if (!name || isNaN(score) || isNaN(weight)) {
    alert('Please fill in all fields correctly');
    return;
  }
```

```
  // Create grade object
  const grade = {
    id: Date.now(),
    name: name,
    score: score,
    weight: weight
  };

  // Add to array
  grades.push(grade);

  // Clear inputs
  nameInput.value = '';
  scoreInput.value = '';
  weightInput.value = '';

  // Update display
  displayGrades();
  calculateFinalGrade();
}
```

```
// Delete a grade
function deleteGrade(id) {
  grades = grades.filter((grade) => grade.id !== id);
  displayGrades();
  calculateFinalGrade();
}
```

```
// Convert percentage to letter grade
function getLetterGrade(percentage) {
  if (percentage >= 90) return "A";
  if (percentage >= 80) return "B";
  if (percentage >= 70) return "C";
  if (percentage >= 60) return "D";
  return "F";
}
```

# High-Quality Software Product

To build high-quality software product, we need to build software that is easy to extend and fix bugs.

- However, using vanilla JavaScript, it is sometime hard to accomplish this goal.
- This is where frameworks and libraries come in to help.

However, in most cases, novice software engineers just use HTML/JavaScript to write logic and UI at the same time.

## UI Logic + Application Logic

Displaying information logic is intermingled with Application Logic.

1. Access the information from HTML using ID.
2. Store the information back to the HTML.

```
// Display all grades
// (Anti-pattern: building HTML strings from user input with innerHTML is unsafe)
function displayGrades() {
  const gradesList = document.getElementById("grades-list");

  gradesList.innerHTML = grades
    .map(
      (grade) => `<div class="grade-item">
            <strong>${grade.name}</strong>:
            ${grade.score}% (Weight: ${grade.weight}%)
            <button onclick="deleteGrade(${grade.id})">Delete</button>
        </div>
    `
    )
    .join("");
}
```

# Calculate final grade: Application logic + UI logic combined

```
function calculateFinalGrade() {
  const resultDiv = document.getElementById("result");

  if (grades.length === 0) {
    resultDiv.innerHTML = "No grades yet";
    return;
  }

  // Calculate weighted average
  const totalWeight = grades.reduce((sum, grade) => sum + grade.weight, 0);

  if (totalWeight === 0) {
    resultDiv.innerHTML = "Total weight must be greater than 0";
    return;
  }
```

```
const weightedSum = grades.reduce((sum, grade) => {
  return sum + grade.score * grade.weight;
}, 0);

const finalGrade = weightedSum / totalWeight;
const letterGrade = getLetterGrade(finalGrade);

resultDiv.innerHTML = `
Final Grade: ${finalGrade.toFixed(2)}% (${letterGrade})
      <br>
      <small>Total weight: ${totalWeight}%</small>
  `;
}
```

## Event Handler

Add grade when Enter is pressed:

```
// Keyboard support
document.addEventListener("DOMContentLoaded", () => {
  const inputs = document.querySelectorAll("input");
  inputs.forEach((input) => {
    input.addEventListener("keydown", (e) => {
      if (e.key === "Enter") {
        addGrade();
      }
    });
  });
});
```

# Pain Points

Look at the code we just wrote:

- `calculateFinalGrade` reads and writes `#result`: it cannot be tested without a browser.
- After every change we must call `displayGrades()` **and** `calculateFinalGrade()`. Forget one, and the screen is stale.

- The logic depends on HTML ids, `onclick` strings, and global functions.
- HTML is built as strings with `innerHTML` (hard to maintain, unsafe for user input).

# The Solution

The `code` directory has the revised version to solve this issue.

## Separate the Calculation

In the `code` directory version, the calculation is a **pure function**: it takes data and returns a result, and it does not touch the DOM.

```
// Pure calculation: it can be tested without a browser or DOM.
function calculateFinalGrade(items) {
  const totalWeight = items.reduce((sum, grade) => sum + grade.weight, 0);
  if (totalWeight === 0) return null;

  const weightedSum = items.reduce(
    (sum, grade) => sum + grade.score * grade.weight,
    0
  );
  return { percentage: weightedSum / totalWeight, totalWeight };
}
```

## Test Without a Browser

```
calculateFinalGrade([
  { score: 100, weight: 50 },
  { score: 80, weight: 50 },
]);
// { percentage: 90, totalWeight: 100 }
```

The same 100 (50%) and 80 (50%) example from before, with no HTML, no `document`, and no buttons.

## One `render()` Draws the Screen

Every change calls `render()` once, instead of calling `displayGrades()` and `calculateFinalGrade()`.

```
function render() {
  gradesList.replaceChildren();
  grades.forEach((grade) => {
    const item = document.createElement('div');
    item.className = 'grade-item';
    const label = document.createElement('span');
    label.textContent = `${grade.name}: ${grade.score}% (Weight: ${grade.weight}%)`;
    const button = document.createElement('button');
    button.textContent = 'Delete';
    button.addEventListener('click', () => deleteGrade(grade.id));
    item.append(label, button);
    gradesList.append(item);
  });

  const summary = calculateFinalGrade(grades); // the pure function
  result.textContent = summary
    ? `Final Grade: ${summary.percentage.toFixed(2)}% (${getLetterGrade(summary.percentage)})`
    : 'Add grades to see your final grade.';
}
```

# What Is Still Left?

Also fixed in this version: `addEventListener` instead of `onclick` strings, and `textContent` instead of `innerHTML`.

Better, but not solved:

- We must still remember to call `render()` after every change.
- `render()` rebuilds the DOM by hand, and the logic still depends on HTML ids.

# Question

- We separated the calculation, but the UI is still updated by hand.
- Where should the state live? How can the UI update automatically when the state changes?

- Ask an LLM, then judge its answer with these criteria:
    i. Can the calculation be tested without a browser?
    ii. Can the UI change without touching the logic?
    iii. Is there a single source of truth for the state?

# What Comes Next

1. **TypeScript example**: group the data and logic in a class (better, but it still touches the DOM)
2. **React example**: the UI is a function of the state